

DiscForge CHD

Conversão e otimização de discos

Aplicativo Windows x64 para imagens PS1/PS2 em CHD.

Base desta apresentação

Código e documentação de main, revisão 34a44a1, consultados em 21/09/2026.

Leitura do material

Recursos atuais, inferências de público e propostas de evolução aparecem identificados.

Problema e oportunidade

Problema inferido

Converter coleções exige lidar com formatos, faixas de áudio, parâmetros e saídas incompletas.

Oportunidade

Reunir extração, identificação, conversão e verificação em uma interface desktop.

Valor esperado

Reduzir trabalho manual e proteger as entradas. Economia de espaço e tempo depende da mídia e dos codecs.

Limite da evidência

O repositório não apresenta pesquisa com usuários nem benchmark de jogos completos.

Visão e objetivos

Visão inferida	Facilitar o preparo e a organização de imagens de discos em CHD no Windows.
Objetivo atual	Processar lotes PS2 e otimizar CHDs de CD para PS1 com verificação antes da saída definitiva.
Objetivos sugeridos	Tornar regressões reproduzíveis e ampliar a confiança na retomada e na distribuição.
Como medir	Taxa de sucesso, falhas por tipo de mídia, tempo por GiB e relação entre tamanho final e inicial. Sem metas numéricas validadas.

Público e personas

Pessoa que organiza imagens PS2

Precisa processar arquivos avulsos e compactados, reconhecer jogos e acompanhar o lote.

Pessoa que mantém CHDs PS1

Quer testar nova compressão e conservar o original quando ele for menor e elegível.

Mantenedor do aplicativo

Precisa reproduzir defeitos, compilar e publicar versões consistentes.

Validação sugerida

Entrevistar usuários sobre formatos, tamanho das coleções e falhas frequentes. Não há personas pesquisadas no repositório.

Funcionalidades disponíveis

Conversão PS2

CD/DVD, imagens avulsas, compactados suportados e CUE multifaixa. Escolha de codecs, hunk e threads.

Otimização PS1

CHDs de CD na pasta escolhida. Pode conservar o CHD original quando menor ou igual ao candidato.

Informações do jogo

Consulta PS2 por serial em Redump e PSX Data Center. Capas PS2 com cache local.

Controle do lote

Progresso, logs, retomada, parada após a entrada atual e exclusão opcional de compactados após sucesso integral.

Escopo e limites

PS1 e PS2 têm escopos distintos

PS1 seleciona apenas CHDs na pasta inicial. PS2 percorre subpastas e inclui compactados.

CUE é obrigatório para faixas agrupadas

BINs numerados sem o CUE original geram erro. O programa evita tratar faixas como discos independentes.

Deteção tem fallback heurístico

ISO sem identificação usa o limite de 400 MiB para CD/DVD. Isso não comprova a mídia real.

Retomada não equivale a verify

Saídas existentes passam por chdman info. CHDs novos passam por chdman verify antes da publicação.

Arquitetura e stack

Interface e aplicação

C# com Windows Forms e .NET Framework, recomendado 4.8.
Executável Windows x64 com instância única.

Motor de processamento

Engine cria ConversionSession. A sessão coordena arquivos, compactados, identificação, conversão e relatórios.

Adaptadores e serviços

ToolRunner executa chdman/7-Zip. Media, Archives e Storage tratam entradas. Services consulta dados e capas.

Persistência e build

Arquivos locais, cache JSON e logs. compilar.ps1 usa csc.exe e incorpora chdman.exe, 7z.exe e 7z.dll.

Fluxo principal de conversão

01 Configuração

Selecionar pastas, plataforma, codecs, hunk e threads. Validar caminhos e parâmetros.

02 Preparação

Enumerar entradas, examinar compactados e CUE, checar saídas existentes e extrair quando necessário.

03 Conversão

Identificar CD/DVD, extrair CHD quando aplicável e executar createcd ou createdvd.

04 Conclusão

Copiar para temporário no destino, executar verify e mover sem sobrescrever. Registrar resultado e limpar temporários.

Casos de uso e exceções

UC01 Converter lote PS2

Ator seleciona imagens/compactados. Resultado: CHDs verificados. Falha de uma entrada gera log e o lote continua.

UC02 Otimizar CHD PS1

Ator fornece CHD de CD. O motor compara os tamanhos e verifica o candidato escolhido. CHD de DVD gera erro.

UC03 Retomar processamento

Saídas esperadas e legíveis permitem ignorar a entrada. CHD existente inválido fica preservado para revisão.

UC04 Interromper ou limpar

Parada respeita a entrada atual. Exclusão habilitada só remove o compactado após todos os discos novos passarem pelas verificações.

User stories de conversão

US01 Conversão PS2

Como usuário PS2, quero converter um CUE com suas faixas, para manter o disco completo em um CHD.

Aceitação US01

Dado CUE válido com dados e áudio, quando converter, então gerar um CHD com as faixas e verify aprovado. Sem CUE, rejeitar BINs agrupados.

US02 Otimização PS1

Como usuário PS1, quero comparar a nova compressão com meu CHD, para evitar uma saída maior.

Aceitação US02

Dado CHD de CD, se o original for menor ou igual ao candidato, então usá-lo e verificá-lo antes da saída final. Rejeitar CHD de DVD.

User stories de segurança

US03 Retomada

Como usuário, quero ignorar saídas já reconhecidas, para evitar extração e conversão repetidas.

Aceitação US03

Com todas as saídas esperadas aprovadas por info, não extrair novamente. Preservar compactado ignorado e saída inválida.

US04 Exclusão opcional

Como usuário, quero apagar compactados após sucesso integral, para liberar espaço com controle.

Aceitação US04

Excluir somente com opção ativa, todos os discos novos verificados e snapshots inalterados. Em falha, preservar a entrada. Não excluir imagens avulsas.

Requisitos funcionais

RF01–RF02 Configurar e selecionar

Validar pastas, plataforma, codecs, hunks e threads. Selecionar entradas conforme PS1/PS2.

RF03–RF04 Preparar e converter

Ler compactados e referências CUE. Converter CD/DVD com chdman e tratar erros por entrada.

RF05–RF06 Validar e retomar

Verificar novas saídas antes do nome definitivo. Detectar saídas existentes sem sobrescrevê-las.

RF07–RF09 Informar e controlar

Consultar serial/capas, exibir progresso e log, parar entre entradas e condicionar a exclusão ao sucesso integral.

Requisitos não funcionais

Integridade e segurança atuais

Saída temporária no destino, verify e movimentação sem sobrescrita. Validação de caminhos e bloqueio de links/junctions.

Resiliência atual

Falhas online não impedem conversão. Cache e limites de rede reduzem dependência dos provedores.

Compatibilidade atual

Windows x64 e .NET Framework. O chdman incluído é descrito como compilado para Zen 3/x64. Ampliar testes de CPUs.

Metas propostas

Medir desempenho por mídia, testar DPI/teclado e definir orçamento de disco temporário. Ainda não há SLA ou benchmark abrangente.

Forma de trabalho ágil

Kanban como ponto de partida

Backlog, Pronto, Em andamento, Revisão/QA e Concluído. Limitar trabalho em andamento a um item por responsável.

Priorização

Primeiro integridade e regressões, depois compatibilidade e conveniência. Ordenar por impacto, risco e esforço.

Cadência sugerida

Triagem semanal, revisão de entregas quinzenal e retrospectiva mensal. Registrar impedimentos quando surgirem.

Scrum como alternativa

Se houver equipe estável, adotar sprints de duas semanas com objetivo, planejamento, revisão e retrospectiva. Papéis ainda precisam de definição.

Definition of Ready e Done

Ready • necessidade clara

Story com ator, valor e critérios testáveis. Identificar módulo afetado e comportamento atual.

Ready • execução viável

Fixture permitida disponível, dependências mapeadas, risco de arquivos avaliado e item pequeno o suficiente.

Done • qualidade demonstrada

Build aprovado, critérios atendidos, regressões relevantes executadas e evidências registradas. Revisar preservação de entradas.

Done • entrega sustentável

Documentação atualizada. Para release do app: versões sincronizadas, changelog, hash e tag. Documentação isolada segue commit próprio.

Backlog priorizado

P0 Regressão reproduzível	Versionar geradores de fixtures sintéticas e runner. Aceite: reproduzir os cenários de TESTES.txt sem coleção pessoal.
P0 Integridade na retomada	Avaliar verify opcional para saídas existentes. Aceite: corrupção de dados detectada em fixture e origem preservada.
P1 Compatibilidade e identificação	Testar CPUs e permitir revisão explícita da mídia heurística. Aceite: matriz documentada e casos ambíguos cobertos.
P1/P2 Distribuição e experiência	P1: pipeline de build/QA e revisão de licenças. P2: benchmark e acessibilidade. Refinar esforço antes de comprometer prazos.

Métricas de acompanhamento

Fluxo de entrega

Lead time, cycle time, throughput e idade dos itens bloqueados. Usar dados do quadro, sem comparar produtividade individual.

Qualidade

Regressões por release, taxa de testes aprovados e falhas por mídia. Separar defeitos do app de entradas inválidas.

Eficiência do produto

Tempo por GiB, tamanho CHD/entrada e pico de espaço temporário em fixtures comparáveis. Registrar CPU e codecs.

Decisão de produto

Estabelecer baseline antes das metas. Rever tendências a cada entrega e priorizar o maior risco observado.

Estratégia de testes e QA

Registro existente

TESTES.txt relata PS1/PS2, ZIP/7z, CUE multifaixa, retomada, parada e exclusão. Relata cinco CHDs iguais à versão anterior por SHA-256.

Limites do registro

São resultados documentados, não reexecutados nesta análise. Não há suíte/fixtures versionadas nem cobertura de todos os formatos ou nova consulta online.

Automação proposta

Testes de caminhos, serial, CUE e parâmetros. Integração com chdman/7-Zip usando fixtures sintéticas e round-trip com hash.

QA manual proposto

Rede indisponível, pouco espaço, colisões, falha de ferramenta, cache corrompido, DPI e navegação por teclado. Gate: nenhuma regressão de integridade.

Build, publicação e CI/CD

Atual • build local

compilar.ps1 compila os fontes com csc.exe, inclui ferramentas e gera o executável.

Atual • publicação local

Publicar.ps1 compila, calcula SHA-256, cria commit/tag e faz push atômico. Não há .github/workflows na revisão analisada.

Proposta • integração contínua

Runner Windows para build e regressões. Reter logs e artefatos, sem jogos nem credenciais no repositório.

Proposta • entrega de releases

Após QA, publicar artefatos e hash por tag. A atualização automática do aplicativo não aparece implementada. O script atual não inclui docs/ no git add.

Riscos e mitigação

Integridade e disco temporário

Falha, pouco espaço ou corrupção podem interromper o lote. Manter verificação atual e adicionar testes de falta de espaço e retomada rigorosa.

Identificação e provedores

Heurística de 400 MiB e alterações no HTML externo podem afetar resultados. Testar parsing e permitir revisão de mídia.

Binários e compatibilidade

Dependências embarcadas exigem validação de origem, licenças e CPUs. Documentar versões e testar atualização controlada.

Manutenção

Classes parciais compartilham estado e testes não estão automatizados. Priorizar fixtures, interfaces de ferramentas e revisão das transições de estado.

Roadmap e próximos passos

Agora • confiança

Confirmar backlog com o mantenedor, registrar baseline e tornar testes de integridade reproduzíveis.

Depois • distribuição

Adicionar CI Windows, documentar dependências/licenças e validar compatibilidade. Gate: regressões aprovadas.

Em seguida • experiência

Avaliar identificação manual, acessibilidade e benchmarks. Priorizar com evidência de uso.

Manutenção contínua

Triar bugs com versão/log sanitizado, revisar provedores e dependências, preservar changelog e revalidar documentação a cada mudança.

Fontes e rastreabilidade

Escopo e operação	README.md e LEIA-ME.txt. Funcionalidades, restrições de plataforma, formatos e dependências.
Implementação	fontes/Processing, Configuration, Storage, Services e Application. Arquitetura e regras conferidas no código.
Qualidade e distribuição	TESTES.txt, PUBLICACAO.md, Publicar.ps1 e fontes/compilar.ps1. Resultados declarados e processo de publicação.
Como interpretar	Links por slide nas notas do PPTX. Personas são inferências. Processo ágil, backlog, metas e roadmap são propostas, sem alegação de implementação.